



Antigravity Directive: ElementNFT V1 Upgrade (IPFS & Variant Randomizer)

Context: We are upgrading the ElementNFT.sol smart contract for the V1 mainnet launch. We need to completely remove the old on-chain SVG generation logic. Instead, the contract will point to a dynamically generated IPFS URI.

Crucial Logic Change: The initial mint (mintElement) MUST always return a Tier 1 (Lead) element to preserve the crafting mechanics. However, we want to pseudo-randomly select which of the 6 visual variants the user receives.



Task 1: Rewrite ElementNFT.sol

Please completely overwrite the existing ElementNFT.sol contract with the following logic.

Key Requirements:

1. **Remove all SVG logic:** Delete the base64 encoding and on-chain drawing functions.
2. **Minting Logic (mintElement):**
 - Requires a 0.002 ETH mint fee.
 - Hardcodes the Tier to 1 (Lead).
 - Uses `uint256(keccak256(abi.encodePacked(block.timestamp, block.prevrandao, msg.sender, tokenId)))` to generate a pseudo-random hash.
 - **Variant Roll:** Randomly assigns a visual variant from 1 to 6 using `(hash % 6) + 1`.
3. **Dynamic tokenURI:** Constructs the metadata URL based on the tier and variant, pointing to a Master IPFS CID (e.g., `ipfs://MASTER_CID/lead/3.json`).

The Required Code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
```

```
import "@openzeppelin/contracts/token/ERC721/extensions/ERC721Enumerable.sol";
import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/utils/Strings.sol";
```

```
contract ElementNFT is ERC721Enumerable, Ownable {
    using Strings for uint256;
```

```
    uint256 private _nextTokenId = 1;
    uint256 public mintFee = 0.002 ether;
```

```
    // IPFS Master CID Base URI (e.g., ipfs://bafybeia.../)
```

```

string public baseURI;

// Mapping from tokenId to its Tier (1 = Lead, 2 = Silver, 3 = Gold)
mapping(uint256 => uint8) public tokenTier;

// Mapping from tokenId to its visual variant (1 through 6)
mapping(uint256 => uint8) public tokenVariant;

event ElementMinted(address indexed minter, uint256 tokenId, uint8 tier, uint8 variant);

constructor(string memory initialBaseURI) ERC721("Alchemy Guild Element", "AGE")
Ownable(msg.sender) {
    baseURI = initialBaseURI;
}

function setBaseURI(string memory _newBaseURI) external onlyOwner {
    baseURI = _newBaseURI;
}

function setMintFee(uint256 _newFee) external onlyOwner {
    mintFee = _newFee;
}

/**
 * @notice Mints a Tier 1 Element with a random visual variant
 */
function mintElement() external payable {
    require(msg.value >= mintFee, "Insufficient ETH for materials fee");

    uint256 tokenId = _nextTokenId++;

    // 1. Generate Pseudo-Random Hash
    uint256 randomHash = uint256(keccak256(abi.encodePacked(
        block.timestamp,
        block.prevrandao,
        msg.sender,
        tokenId
    )));

    // 2. Hardcode Tier to 1 (Lead)
    uint8 tier = 1;

    // 3. Determine Variant (Roll 1-6)

```

```

uint8 variant = uint8((randomHash % 6) + 1);

// 4. Save State & Mint
tokenTier[tokenId] = tier;
tokenVariant[tokenId] = variant;

_safeMint(msg.sender, tokenId);

emit ElementMinted(msg.sender, tokenId, tier, variant);
}

// NOTE: Ensure your existing `ascendElement` function is preserved or updated
// to correctly assign Tier 2 and Tier 3 when lower tiers are burned!

function tokenURI(uint256 tokenId) public view override returns (string memory) {
    require(_ownerOf(tokenId) != address(0), "URI query for nonexistent token");

    uint8 tier = tokenTier[tokenId];
    uint8 variant = tokenVariant[tokenId];

    string memory tierFolder;
    if (tier == 1) tierFolder = "lead";
    else if (tier == 2) tierFolder = "silver";
    else if (tier == 3) tierFolder = "gold";

    // Concatenate: baseURI + tierFolder + "/" + variant + ".json"
    return string(abi.encodePacked(
        baseURI,
        tierFolder,
        "/",
        uint256(variant).toString(),
        ".json"
    ));
}

function getTokenTier(uint256 tokenId) external view returns (uint8) {
    return tokenTier[tokenId];
}

function withdrawFees() external onlyOwner {
    payable(owner()).transfer(address(this).balance);
}
}

```

Output Requirement

Please implement this new contract logic. Be sure to integrate this new minting and URI logic with the existing ascendElement function so that crafting still works correctly.